

Tutorial: Kruskal's and Prim's Algorithms

Minimum Spanning Tree concepts with worked examples

Minimum Spanning Tree (MST): For a connected, undirected, weighted graph, an MST is a spanning tree that includes all vertices, has no cycles, and has the minimum possible total edge weight.

1. Kruskal's Algorithm

Kruskal's algorithm builds the MST by selecting the smallest edges first while avoiding cycles.

Main idea

- Sort all edges in non-decreasing order of weight.
- Pick the smallest edge that does not create a cycle.
- Continue until exactly $V - 1$ edges are chosen.

To check whether an edge creates a cycle, Kruskal's algorithm commonly uses a Disjoint Set (Union-Find) data structure.

Worked Example for Kruskal

Consider the graph with vertices A, B, C, D and the following weighted edges:

Edge	Weight
A-B	1
B-C	2
A-C	3
B-D	4
C-D	5
A-D	6

Edges sorted by weight: A-B, B-C, A-C, B-D, C-D, A-D.

Step	Chosen Edge	Decision
1	A-B = 1	Include
2	B-C = 2	Include
3	A-C = 3	Reject (forms cycle A-B-C)
4	B-D = 4	Include

MST obtained by Kruskal: {A-B, B-C, B-D}

Total cost = $1 + 2 + 4 = 7$

2. Prim's Algorithm

Prim's algorithm starts from one vertex and grows a single tree by repeatedly adding the minimum weight edge that connects the current tree to a new vertex.

Main idea

- Start from any one vertex.
- Among all edges from the visited set to the unvisited set, choose the smallest.
- Add the new vertex and edge to the MST.
- Repeat until all vertices are included.

Worked Example for Prim

Use the same graph and start from vertex A.

Step	Visited Vertices	Candidate Edges	Chosen Edge
1	{A}	A-B=1, A-C=3, A-D=6	A-B = 1
2	{A, B}	B-C=2, B-D=4, A-C=3, A-D=6	B-C = 2
3	{A, B, C}	B-D=4, C-D=5, A-D=6	B-D = 4

MST obtained by Prim: {A-B, B-C, B-D}

Total cost = $1 + 2 + 4 = 7$

3. Kruskal vs Prim

Feature	Kruskal	Prim
Starting point	Works with edges globally	Starts from one vertex
Main strategy	Select smallest edge without cycle	Grow one connected tree
Common data structure	Union-Find	Min-priority queue
Typical complexity	$O(E \log E)$	$O(E \log V)$ with heap
Often preferred for	Sparse graphs	Dense graphs

4. Time Complexity

- Kruskal's algorithm: $O(E \log E)$, mainly because of edge sorting.
- Prim's algorithm using adjacency matrix: $O(V^2)$.
- Prim's algorithm using adjacency list and min-heap: $O(E \log V)$.

5. Definitions

Kruskal's Algorithm: A greedy algorithm that finds an MST by selecting edges in increasing order of weight without forming cycles.

Prim's Algorithm: A greedy algorithm that finds an MST by starting from a vertex and repeatedly adding the minimum weight edge that connects the tree to a new vertex.

6. Key Memory Trick

Kruskal → choose smallest edge globally | Prim → grow one tree from a start vertex